RTS3917 Series SDK User Guide

Release v5.3

Realtek

Oct 09, 2025

COPYRIGHT

DISCLAIMER

Realtek provides this document ``as is'', without warranty of any kind, neither expressed nor implied, including, but not limited to, the particular purpose. Realtek may make improvements and/or changes in this document or in the product described in this document at any time. This document could include technical inaccuracies or typographical errors.

TRADEMARKS

Realtek is a trademark, of Realtek Semiconductor Corporation. Other names mentioned in this document are trademarks/registered trademarks of their respective owners.

USING THIS DOCUMENT

This document is intended for the software and firmware engineer's general information on the Realtek IP Camera IC. Though every effort has been made to ensure that this document is current and accurate, more information may have become available subsequent to the production of this guide. In that event, please contact your Realtek representative for additional information that may help in the development process.

Product Version

The SDK version corresponding to this document:

SDK Version
SDK v5.3 - SDK v5.3.x

Content

Image

Table

1 SDK Introduction

1.1 RTS3917 Series Chip Introduction

The RTS3917 series maily consists of three SoCs: RTS3917N, RTS3917 and RTS3918N.

The relevant information is shown in table *RTS3917 Series Chip Introduction*.

Table 1.1: RTS3917 Series Chip Introduction

Model	RTS3917N	RTS3917	RTS3918N
CPU	ARM Cortex A7	ARM Cortex A7	ARM Cortex A7
DDR	Built-in 512Mbit DDR2	Built-in 512Mbit DDR2	Built-in 1Gbit DDR3
H.264/H.265	5M@30fps	5M@30fps	8M@30fps
NPU	————	————	————
wrap	QFN88	BGA173	QFN88
Sensor Input	Single MIPI or DVP		
Encoding Support	MJEPG, H264, H265		
streaming Support	4 * YUV outputs and 1 * RGB output in parallel.		
Ethernet	EPHY		
interfaces	USB2.0. SDIO*2. PWM*4. UART*2. SPI etc		

1.2 directory structure

The Realtek RTS3917 Platform SDK is a Buildroot-based embedded Linux build and development framework used to provide an integrated development environment for the Realtek RTS3917 platform SDK is an embedded Linux build and development framework based on Buildroot to provide an integrated development environment for the RTS39XX series chips. The framework fully utilizes the BR2_EXTENAL mechanism of Buildroot. Therefore, it is recommended that readers understand the principles and usage of Buildroot before reading this article. The Buildroot user manual can be found at https://buildroot.org/downloads/manual/manual.html.

The directory structure of the SDK is as follows:

```
|-- rts39xx_sdk_va.b.c(a.b.c is the SDK version number)
|   |-- buildroot-dist          => Buildroot source code
|   |-- dl                      => Basic open source package
|   |-- launch.sh               => Compile Scripts
```

(continues on next page)

(continued from previous page)

```
|   |-- platform
|   |   |-- board               => Board Support Documentation
|   |   |-- common.mk
|   |   |-- Config.in           => Extended Menu Entry
|   |   |-- configs             => Buildroot Default Configuration
|   |   |-- external.desc       => BR2_EXTENAL Description file
|   |   |-- external.mk         => BR2_EXTENAL makefile entry
|   |   |-- local.mk
|   |   |-- package             => Package Menu File
|   |   |-- patches             => Package Patch File
|   |   |-- scripts             => Auxiliary Scripts
|   |   |-- source
|   |   |   |-- bootloader      => bootloader source code
|   |   |   |   |-- uboot-2023.01
|   |   |   |-- debug           => Disk Performance Test Tool
|   |   |   |-- drivers         => Modular drive
|   |   |   |-- graphic         => Graphical User Interface Support System
|   |   |   |-- kernel          => Kernel source code
|   |   |   |   |-- linux-6.6
|   |   |   |-- lib             => open source runtime library
|   |   |   |-- network         => Software for configuring the network
|   |   |   |-- rtscore         => API library files and sample code
|   |   |   |-- system          => open source software
|   |   |   |-- usb             => uvc drawing application
|   |-- toolchain               => development toolchain
```

2 Environment Setup and Compilation

2.1 Prepare the Development Environment

It is recommended to choose a mainstream Linux distribution as the development environment, as it makes it easier to find various technical resources and support.

This document will use Ubuntu Desktop 18.04 x64 LTS as an example for the following introduction.

On the Ubuntu 18.04 x64 version, the user needs to install the following necessary packages:

```
$ sudo apt-get install build-essential libncurses5-dev libc6:i386 git
```

Users of other distributions can refer to Buildroot Chapter II of the manual (https://buildroot.org/downloads/manual/manual.html#requirement) Install the necessary packages for the system.

2.2 SDK extraction

The steps to install the RTS3917 SDK on the compilation host are as follows:

1. Copy

   Copy the rts39xx_sdk_va.b.c.tar.bz2.gpg (where a.b.c is the SDK version number) to the compilation host.

2. Decrypt & verify the signature

   If decryption and signature verification are not required, you can directly skip to Step 3: Extraction.

   On the compilation host, make sure the gpg tool is available. If not, install it via *sudo apt-get install gnupg2*.

   Step 1: Obtain the key with the fingerprint A203D2496E6BED61B1C055C6291E3FB529CA599A.

   ```
   $ gpg2 --keyserver keyserver.ubuntu.com --recv-keys␣
   ↪A203D2496E6BED61B1C055C6291E3FB529CA599A
   ```

   or

   ```
   $ gpg2 --keyserver keyserver.ubuntu.com --locate-keys pclinux@realsil.com.cn
   ```

   Note: Verifying the signature is intended to help the customer confirm the integrity of the SDK they received. This operation is not mandatory. That is, even if the user does not perform Step 1 to import the key with the fingerprint A203D2496E6BED61B1C055C6291E3FB529CA599A, they can still successfully obtain the decrypted file rts39xx_sdk_va.b.c.tar.bz2 in Step 2. Customers can choose to either verify the signature or skip this step based on their needs.

   Step 2: Decrypt & Verify Signature

```
$ gpg2 -o rts39xx_sdk_va.b.c.tar.bz2 -d rts39xx_sdk_va.b.c.tar.bz2.gpg (a.b.
↪c is SDK version)
```

After entering the command, a prompt will appear asking for the correct decryption password to extract the file. Once the archive is decrypted, you will obtain the rts39xx_sdk_va.b.c.tar.bz2 (where a.b.c represents the SDK version number) SDK source code package.

a). If the customer has performed the above-mentioned Step 1 and imported the related key, the decryption and signature verification will be carried out. The log will appear as follows:

```
$ gpg2 -o rts39xx_sdk_v4.1_rc2_20200818.tar.bz2 -d rts39xx_sdk_v4.1_rc2_
↪20200818.tar.bz2.gpg
  gpg: AES256 encrypted data
  gpg: encrypted with 1 passphrase
  gpg: Signature made Tue Sep 8 12:40:10 2020 CST
  gpg: using RSA key A203D2496E6BED61B1C055C6291E3FB529CA599A
  gpg: Good signature from "Realsil PC Linux <pclinux@realsil.com.cn>"␣
↪[marginal]
  gpg: pclinux@realsil.com.cn: Verified 2 signatures in the past 97 minutes.
  Encrypted 0 messages.
  gpg: Warning: you have yet to encrypt a message to this key!
  gpg: Warning: if you think you\'ve seen more signatures by this key and␣
↪user
  id, then this key might be a forgery! Carefully examine the email address
  for small variations. If the key is suspect, then use
  gpg --tofu-policy bad A203D2496E6BED61B1C055C6291E3FB529CA599A
  to mark it as being bad.
  gpg: WARNING: This key is not certified with sufficiently trusted␣
↪signatures!
  gpg: It is not certain that the signature belongs to the owner.
  Primary key fingerprint: A203 D249 6E6B ED61 B1C0 55C6 291E 3FB5 29CA 599A
```

b). If the customer has not performed the above-mentioned Step 1, meaning they have not imported the related key, a signature verification error will occur. However, this error will not affect the decryption process. The log will appear as follows:

```
$ gpg2 -o rts39xx_sdk_v4.1_rc2_20200818.tar.bz2 -d rts39xx_sdk_v4.1_rc2_
↪20200818.tar.bz2.gpg
  gpg: AES256 encrypted data
  gpg: encrypted with 1 passphrase
  gpg: Signature made Tue Sep 8 11:00:30 2020 CST
  gpg: using RSA key A203D2496E6BED61B1C055C6291E3FB529CA599A
  gpg: Can't check signature: No public key
```

3. Extract

   Use the command: *tar -xjvf rts39xx_sdk_va.b.c.tar.bz2* Extract the compressed file to get the rts39xx_sdk_va.b.c directory.

2.3 SDK Compilation

In the extracted root directory rts39xx_sdk_va.b.c, select the configuration to be compiled through the launch.sh script.

There are two ways to run the launch.sh script.

1. Interactive mode, recommended for use.

```
$ ./launch.sh
1) rts3917n_base_defconfig
2) rts3917_base_defconfig
3) rts3918n_base_defconfig
Type a number or 'q' to quit:
```

2. Directly specify the defconfig to be compiled.

Check which defconfig files are provided by the SDK.

```
$ ls platform/configs/
rts3917n_base_defconfig   rts3918n_base_defconfig   rts3917_base_defconfig
```

The defconfig file is a pre-saved Buildroot menu configuration file. Taking `rts3918n_base_defconfig` as an example.

```
$ ./launch.sh rts3918n_base_defconfig
```

By default, launch will create a workspace directory named out/rts3918n_base. If you want to specify a different directory name, you can add extra parameters, for example:

```
$ ./launch.sh rts3918n_base_defconfig myfolder
```

It will create a workspace directory named out/myfolder.

After running launch, the compilation workspace is created.

```
$ cd out/rts3918n_base
$ ls
build  Makefile  suitcase
```

In the defconfig file, the default selected sensor is the SC3235 sensor

Before starting the compilation, you can use the command *make menuconfig* to inspect the configuration menu. After reviewing the settings, you can proceed with the compilation by executing the command *make* .

The compilation time depends on your machine's performance. If everything goes smoothly, once the compilation is complete, the following image will be generated.

```
$ ls images/
rootfs.squashfs  u-boot.bin  uImage.dtb  userdata.bin
```

If you want to generate the WebGUI upgrade image and the raw Flash image, you will need to unlock the boot and userdata partitions in the raw.ini file. Then, execute the additional command *make pack* to generate the required images.

```
$ make pack
$ ls images
linux.bin  raw_nor.bin  rootfs.squashfs  u-boot.bin  uImage.dtb  userdata.bin
```

2.4 Firmware Burning

If U-Boot is already running on the target board, you can update U-Boot directly by connecting to the server via serial port or network.

If the target board is being used for the first time and U-Boot is not running on the board, U-Boot needs to be written using a programmer or through rescue mode (specific instructions can be found in the user's rescue guide document).

The following operations are based on the assumption that U-Boot is already running on the target board.

2.4.1 Flash the image via TFTP

At the U-Boot stage, you can burn the binary image to Flash via TFTP.

TFTP installation and network configuration.

If the development board only has the U-Boot image and hasn't flashed the correct Linux kernel, it will directly enter the U-Boot command line after powering on. If the correct Linux kernel has been flashed, U-Boot will have a 2-second countdown, and pressing any key on the keyboard before the countdown ends will also allow entering the U-Boot command line.

1. Configure the network under U-Boot

First, you need to confirm whether the network is correctly configured in U-Boot. You can execute the command printenv to check, as shown in Fig. 2.1. Pay attention to the network-related environment variables such as ethaddr, ipaddr, and netmask to ensure they are set correctly.

```
=> printenv
addmisc=setenv bootargs ${bootargs} console=ttyS0,${baudrate} panic=1
baudrate=115200
bootcmd=bootm ${kernel_offset} - ${dtb_addr}
bootdelay=2
bootfile=/vmlinux.elf
dfu_alt_info=0 ram 0x80400000 0x400000
dfu_bufsiz=0x400000
dtb_addr=8775bf00
ethact=r8168#0
ethaddr=00:e0:4c:50:00:46
fdt_high=0xffffffff
hostname=IPCAM-000000
ipaddr=10.0.9.225
kernel_offset=a0000
load=tftp 80500000 ${u-boot}
netmask=255.0.0.0
stderr=serial
stdin=serial
stdout=serial

Environment size: 452/131068 bytes
```

Fig. 2.1: The output of printenv

If the network is not configured correctly, execute the following command to set it:

```
# setenv ethaddr 00:e0:4c:02:00:40
# setenv ipaddr 10.0.2.24
# setenv netmask 255.255.240.0
```

Note: This article assumes that the development board's IP address is 10.0.2.24, and the subnet mask is 255.255.240.0. Therefore, the PC's IP can be set from 10.0.0.1 to 10.0.15.254.

This operation can only temporarily configure the network parameters, and the parameters will be lost after the system is restarted.

If you need to save the configuration information, first execute sf protect unlock, then modify the configuration, and finally use the saveenv command to save the changes

1.Install TFTP client on PC

Here are the recommended versions and usage examples for the TFTP client on Windows and Linux.

- Windows: It is recommended to install tftpd32, official website: http://tftpd32.jounin.net/

  The usage of the tftpd32 tool is as follows: Fig. 2.2 , Host refers to the IP address of the development board. **Port** enter the default port number for TFTP is 69, **Local File** select the local image file to be flashed,To flash the image, simply click the **Put** button.

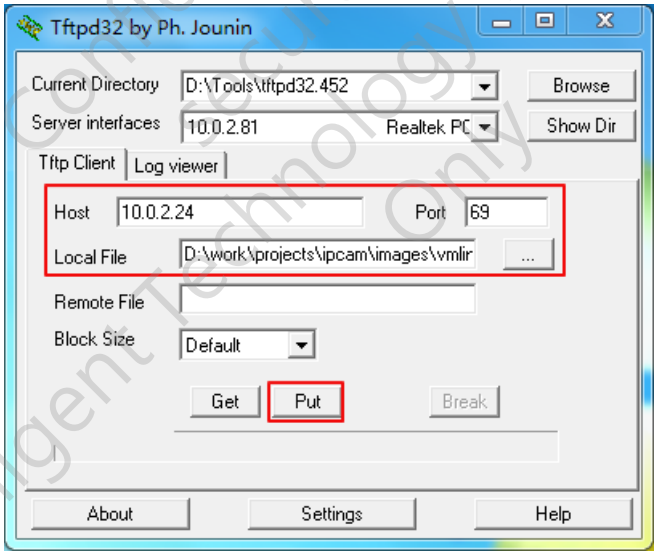

Fig. 2.2: The interface of the tftpd32 tool.

- Linux: It is recommended to install tftp-hpa.

  The command line usage of tftp-hpa is as follows:

```
$ tftp 10.0.2.24 -m binary -c put linux.bin
```

There are differences in the interface between the Windows and Linux clients (one uses a graphical interface, while the other uses a command-line interface), but fundamentally, there is no difference. Therefore, when introducing different methods for flashing the image, only command-line examples will be provided

Flashing U-Boot,kernel,rootfs,linux.bin

When flashing uboot, kernel, rootfs, and linux.bin, you can follow the commands in the table *RTS3917 Partition Firmware Flashing* .

If you need to flash the entire flash, refer to the next section. *Burning the raw_nor.bin*

Table 2.1: RTS3917 Partition Firmware Flashing

Partitions	commands in uboot	commands in PC
uboot	update uboot	tftp 10.0.2.24 -m binary -c put u-boot.bin
kernel	update kernel	tftp 10.0.2.24 -m binary -c put uImage.dtb
rootfs	update rootfs	tftp 10.0.2.24 -m binary -c put rootfs.squashfs
linux.bin	update all	tftp 10.0.2.24 -m binary -c put linux.bin

The above assumes the development board's IP address is 10.0.2.24.

The linux.bin is a binary image generated by *make pack* . It can include one or more partition image files, and the included partitions can be configured through the merge.ini file.

For detailed information about the file structure of linux.bin and flash partitions, please refer to Chapter *Image file structure* .

Burning the raw_nor.bin

raw_nor.bin is a flash binary image generated by *make pack* . It includes the image files of all partitions, and the included partitions can be configured through the raw.ini file. (The following example demonstrates writing raw_nor.bin to a 16M flash.)

For detailed information about the file structure of raw_nor.bin and flash partitions, please refer to Chapter *Image file structure* .

1. Connect the development board and the PC via Ethernet.
2. Execute the command in the U-Boot command line:

```
# tftpsrv 0x80400000
```

3. Execute the TFTP client on the PC side:

```
$ tftp 10.0.2.24 -m binary -c put raw_nor.bin
```

4. After U-Boot receives raw_nor.bin via TFTP, execute the command to flash it:

```
# sf probe 0;
# sf protect unlock;
# sf erase 0x0 0x1000000;sf write 0x80400000 0x00000 0x1000000;
```

2.5 Flash partition adjustment

2.5.1 Partition configuration file specification

The Flash partition is configured through the DTS configuration file. Customers can use the default DTS configuration when first using it, or specify a configuration file in the BuildRoot configuration menu.

```
$ make menuconfig
  Kernel  --->
    [*] Linux Kernel
          Kernel version (Custom version)  --->
    (6.6) Kernel version
    ...
    (realtek/rts3918n_evb) Device Tree Source file names
    ...
```

The SDK provides several default partition configuration files, i.e., DTS files in **kernel/linux-6.6/arch/arm/boot/dts/realtek/** :

```
rts3917n_evb.dts  rts3917_evb.dts rts3918n_nand.dts rts3918n_evb.dts
```

Users can also create their own partition configuration file, such as a new DTS file rts3918n_new.dts, and specify the use of the new configuration file in the BuildRoot configuration menu.

If the partition file is modified and the partitions are adjusted, you need to rebuild U-Boot and the kernel by running make; make uboot-rebuild; make linux-rebuild.Uncomment the U-Boot partition in merge.ini and refer to *RTS3917 Partition Firmware Flashing* to update the linux.bin file. Alternatively, you can directly update the raw_nor.bin.

2.5.2 Modify partitions through the DTS partition configuration file

Typically, the default configuration file will configure the following partitions:

1. boot
2. kernel
3. rootfs
4. userdata

The default partition configuration file is located at the following path:

kernel/linux-6.6/arch/arm/boot/dts/realtek/rts39xx_xxx.dts

Taking rts3918n_evb.dts as an example:

```
partitions {
  compatible = "fixed-partitions";
  #address-cells = <1>;
  #size-cells = <1>;

  global@0x0 {
```

(continues on next page)

(continued from previous page)

```
        label = "global";
        reg = <0x000000 0x1000000>; //flash size, 16M
    };

    partition@0x0 {
        label = "boot";
        reg = <0x000000 0xa0000>; //10*64K for uboot, 128K for hconf
    };

    partition@0x100000 {
        label = "kernel";
        reg = <0x0A0000 0x400000>; //64*64K
    };

    partition@0x400000 {
        label = "rootfs";
        reg = <0x4A0000 0x680000>; //104*64K
    };

    partition@0x6400000 {
        label = "userdata";
        reg = <0xB20000 0x4e0000>; //78*64K
    };
};
```

The Flash partition can be adjusted as needed. Users can either modify the default configuration file provided by the SDK or create their own partition configuration file.

Modifications to the configuration file must follow the following rules:

1. The configuration file is divided into multiple sections, and the “global” section must be placed at the beginning.
2. The “global” section is mainly used to specify the total size of the flash.
3. In the partition section, the “label” field is the partition description, and the “reg” field contains the partition’s starting address and size. The partition size must be allocated in multiples of 64K.
4. The four partitions: boot, dtb, kernel, and rootfs, must exist. The boot partition must be placed at the beginning of the flash, and the names of the dtb, kernel, and rootfs partitions cannot be modified.

The followingThe points following should points also should be also noted be:

1. **U-Boot Partition Location is Fixed.** : The U-Boot partition cannot be relocated. Since U-Boot must run first upon chip power-on, the boot partition must be placed at the very beginning of the Flash. The order of other partitions is adjustable.
2. **Kernel and rootfs Partition Names are Immutable.** : The names of the kernel and rootfs partitions cannot be modified. During execution, U-Boot reads the DTB file to determine the Linux kernel load address and derives the root partition device name (for bootargs) from the rootfs partition name.
3. **If a new partition needs to be added, it can be placed at any location after the rootfs partition.**
4. **After modification, U-Boot and the kernel need to be recompiled and updated.**: After modifying the partition configuration file, recompile using the commands: make, make uboot-rebuild, and make linux-rebuild. After uncommenting U-Boot in the merge.ini file, run make pack to generate and update linux.bin, and the new partition scheme will take effect. Refer to the table in *RTS3917 Partition Firmware Flashing* to update linux.bin or directly update the entire flash by flashing raw_nor.bin.

2.6 Mounting NFS

2.6.1 Setting up NFS server on Ubuntu

Assume the Ubuntu username is 'ipcam', and the NFS shared path is **/home/ipcam/share/nfs** , with the IP address as 10.0.1.10.

- Install the NFS server

```
$ sudo apt-get install nfs-kernel-server
```

- Configure the shared directory by editing /etc/exports.

```
/home/ipcam/share/nfs  *(rw,sync,no_root_squash)
```

- Restart the NFS service.

```
$ sudo service nfs-kernel-server restart
```

2.6.2 Mount the NFS shared directory

- Configure the local network

```
# ifconfig eth0 10.0.1.24
```

- Mount the remote NFS shared directory

```
# mount -t nfs -o nolock 10.0.1.10:/home/ipcam/share/nfs /mnt/
```


2.7 Single package compilation and image reconstruction

2.7.1 Quickly rebuild rootfs and the image

Buildroot is an excellent compilation system, well-suited for building small to medium-sized embedded Linux systems. However, it has some inherent limitations. Since Buildroot does not support the feature of uninstalling individual packages, the correctness of the compilation needs to be considered at least in the following scenarios:

1. After the compilation is complete and the rootfs has been generated, if you try to deselect a package through menuconfig and then run make again, the contents of the package will still remain in the rootfs and will not be removed.
2. A package installs the header file foo.h under staging/usr/include. Later, the package modifies the source code, renaming the original foo.h to bar.h. After recompiling the package, the header file bar.h will be installed under staging/usr/include, but foo.h will still remain in its original location, which could interfere with the compilation of other packages.
3. A package installs the command foo under target/bin. Later, the package modifies the source code and makefile, changing the installation location of foo to target/usr/bin. Since the old version of foo is still present under target/bin, it may have an incorrect impact on the system's operation.

To address these situations, the SDK provides the "rr" build target, which allows for quick reconstruction of the rootfs and image.

```
$ make rr
```

The use of "rr" is based on having performed a full compilation once. Since the package compilation steps are skipped, the time required to rebuild the rootfs and image is significantly reduced, avoiding the need to relaunch and run make for small changes.

2.7.2 Single package compilation

```
$ make <package_name>
```

The package_name is the directory name under buildroot-dist/package/ or platform/package/.

2.7.3 Single package recompilation

```
$ make <package_name>-rebuild
```

The package_name is the directory name under buildroot-dist/package/ or platform/package/. When the source code of a package is modified, it is recommended to use this command. Other subcommands for single package compilation can be referenced. Buildroot manual 8.12.5 (https://buildroot.org/downloads/manual/manual.html#_advanced_usage)

2.7.4 Single package reconfiguration and compilation

```
$ make <package_name>-reconfigure
```

The package_name is the directory name under buildroot-dist/package/ or platform/package/. When the configuration of a package is modified in menuconfig or the package's Makefile, it is recommended to use this command.

3 U-Boot

3.1 U-Boot Introduction

A bootloader is a small program that runs before the Linux kernel runs, this small program is used to initialize hardware devices and boot the OS kernel. The SDK uses U-Boot as the bootloader.

3.1.1 Booting U-Boot

After powering up the development board, the default standard input and standard output of the development board is UART1, connect UART1 on the development board to the PC. The UART is set to:

- Baud rate: 57600
- Data bits: 8
- Parity: None
- Stop bit: 1
- Flow control: none

3.1.2 U-Boot common commands

Note: U-Boot supports auto-completion of commands, when you enter part of a command, press the Tab key, the system will automatically complete or list the possible commands.

Common commands supported by U-Boot are shown in table *U-Boot common command description*.

Table 3.1: U-Boot common command description

command	description
?	Get a list of all commands or list the help for a command. Usage:? [command …] Description:List the help information of the command. When no parameter, list all commands and brief description.
help	help Same as?
printenv	Print environment variables. Usage:printenv [name …] Description:Print environment variables. When no parameter is taken, all variables are printed.
setenv	Set or delete variables. Usage:setenv name [value] Description:name is the name of the U-Boot environment variable, or it can be a user-defined variable. When value is empty, delete variable “name”, otherwise set variable “name” with value.
saveenv	Save variables set under uboot. Usage:saveenv Description:saveenv is to write the environment variables under uboot into Flash, otherwise it is a temporary modification.
ping	Used to simply determine the status of the destination host network or the working status of the local network. Usage:ping <ipaddr> Description: ipaddr denotes the IP address of the destination host, when the network is working normally, the result will show “host <ipaddr> is alive”, Otherwise show “ping failed;host <ipaddr> is not alive”.
tftpsrv	Start tftp server Usage:tftpsrv Description:Start tftp server, load the image file passed by tftp client to a fixed location in memory space.
bootm	Set up the runtime environment and start executing the binary code. Usage:bootm [addr [arg …] Description:Execute the code at addr address, the binary code is required to be the binary file processed by mkimage.
update all	Start the tftp server and burn the received image file to flash at a specific address.

3.2 U-Boot Compilation

3.2.1 Buildroot Integrated Compilation

Execute make menuconfig in the compile workspace to open the Buildroot Configuration Image screen and select U-Boot as the Bootloader:

```
$ make menuconfig
Bootloaders  --->
[ ] afboot-stm32
[ ] AT91 Bootstrap 3
[ ] ARM Trusted Firmware (ATF)
[ ] Barebox
[ ] grub2
[ ] mxs-bootlets
[ ] optee_os
[ ] s500-bootloader
[*] U-Boot
        Build system (Kconfig)  --->
        U-Boot Version (Custom version)  --->
(2023.01) U-Boot version
()    Custom U-Boot patches
($(BR2_EXTERNAL)/package/uboot) Custom U-Boot patches
        U-Boot configuration (Using a custom board (def)config file)  --->
($(BR2_EXTERNAL)/board/rts3917/uboot_$(BR2_TARGET_UBOOT_CUSTOM_VERSION_
↪VALUE)_3918n_defconfig)
()    Additional configuration fragment files
[ ]   U-Boot needs dtc
[ ]   U-Boot needs pylibfdt
[ ]   U-Boot needs pyelftools
[ ]   U-Boot needs OpenSSL
[ ]   U-Boot needs lzop
        U-Boot binary format  --->
[ ]   produce a .ift signed image (OMAP)
[ ]   Install U-Boot SPL binary image
[ ]   Environment image  ----
[ ]   Generate a U-Boot boot script
()    Device Tree Source file paths
()    Custom make options
```

Buildroot compiles U-Boot roughly as follows: Buildroot copies the specified version of U-Boot source code from platform/source/bootloader/uboot-<BR2_TARGET_UBOOT_VERSION> to build/uboot-custom. Then copy the def-config specified by BR2_TARGET_UBOOT_CUSTOM_CONFIG_FILE to build/uboot-custom/.config. Then start the U-Boot cross-compilation.

U-Boot's menu configuration can be viewed and modified with the single-packet subcommand:

```
$ make uboot-menuconfig
```

Note that the uboot-sub command actually occurs in the build/uboot-custom directory, not in the source directory under platform. directory under platform. If the U-Boot configuration menu changes, execute the uboot build target to recompile U-Boot.

```
$ make uboot
```

If you change the configuration via uboot-menuconfig and want to save it for the next compilation, you can use the subcommand uboot-update-defconfig overrides the defconfig specified by BR2_TARGET_UBOOT_CUSTOM_CONFIG_FILE. file specified by BR2_TARGET_UBOOT_CONFIG_FILE.

```
$ make uboot-update-defconfig
```

Recompile U-Boot with the new configuration.

```
$ make uboot-dirclean uboot
```

3.2.2 U-Boot standalone compilation

U-Boot can be compiled independently of the SDK by first creating a separate compilation workspace.

```
$ mkdir uboot_work
$ cd uboot_work
```

Copy the required source code and default configuration from the SDK.

```
$ cp -afr [path/of/rts39xx_sdk_va.b.c/platform/source/bootloader/uboot-2023.
↪01] .
$ cp -f [path/of/rts39xx_sdk_va.b.c/platform/board/rts3917/uboot_2023.01_
↪rts3918n_defconfig uboot-2023.01/.config
```

Setting up the compilation environment

```
$ PATH=$PATH:[path/of/rts39xx_sdk_vx.x.x/toolchain/asdk-12.4.1-a7-EL-6.6-u1.
↪0-a32nh-linux-x86_64-241017/bin]
$ export ARCH=arm
$ export CROSS_COMPILE=arm-linux-
```

compiling

```
$ cd uboot-2023.01
$ make -j4
```

The generated u-boot.bin is located in the U-Boot source root directory.

3.2.3 U-Boot Board Configuration

In the U-Boot configuration menu, select `Board config` to enter the board configuration menu item. In general, the board configuration directly uses the default configuration items.

In the board configuration menu, first of all, you need to select the chip type, RTS3917/RTS3918 series chips, in the `Chip Selection` select `RTS3917`.

Different chip types carry different DDR memories, so pay special attention to the differences in chip models and the capacity of external DDR memories during the U-Boot compilation phase. Depending on the model and capacity of the DDR memory on board, the corresponding U-Boot image is compiled. The specific U-Boot configuration options correspond to the chip model as follows:

The `DDR Type Selection` option for RTS3917/RTS3918 series chips and the corresponding chip types are listed below:

```
QFN88 DDR2 MCM(512Mbit)      => RTS3917N + Built-in 512Mbit DDR2

QFN88 DDR3 MCM(1Gbit)        => RTS3918N + Built-in 1Gbit DDR3

BGA173 DDR3 External         => RTS3917 + Built-in 512Mbit DDR2
```

4 Linux kernel

4.1 Buildroot Integrated Compilation

Note: If you do not have sufficient knowledge of the RTS3917 platform, do not modify the default configuration, but add the required modules.

Execute make menuconfig in the build workspace to open the Buildroot configuration GUI. Linux-related configuration is available in the Kernel submenu. menu.

```
$ make menuconfig
Kernel  --->
    [*] Linux Kernel
        Kernel version (Custom version)  --->
    (6.6) Kernel version
    ()    Custom kernel patches
        Kernel configuration (Using a custom (def)config file)  --->
    ($(BR2_EXTERNAL)/board/rts3917/linux_$(BR2_LINUX_KERNEL_CUSTOM_VERSION_
↪VALUE)_defconfig)
        Configuration file path
    ()    Additional configuration fragment files
    ()    Custom boot logo file path
        Kernel binary format (uImage)  --->
        Kernel compression format (gzip compression)  --->
    ()    load address (for 3.7+ multi-platform image)
    [*]   Build a Device Tree Blob (DTB)
            Device tree source (Use a device tree present in the kernel)  ---
↪>
    (realtek/rts3918n_evb) Device Tree Source file names
    ()      Out-of-tree Device Tree Source file paths
    [ ]     Keep the directory name of the Device Tree
    [ ]     Build Device Tree with overlay support
    [ ]   Install kernel image to /boot in target
    [ ]   Needs host OpenSSL
    [ ]   Needs host libelf
    [ ]   Install kernel image to /boot in target
        Linux Kernel Extensions  --->
        Linux Kernel Tools  --->
```

Buildroot compiles Linux roughly as follows: Buildroot copies the specified version of Linux source code from platform/source/kernel/linux-<BR2_LINUX_KERNEL_VERSION> to build/linux-custom. After that, copy the defconfig specified by linux_6.6_defconfig under /platform/board/rts3917/ to build/linux-custom/.config, and then start the cross-compilation of Linux.

The Linux menu configuration can be viewed and modified with the linux-sub command:

```
$ make linux-menuconfig
```

Note that the linux-subcommand actually occurs in the build/linux-custom directory, not in the source directory under platform. directory under platform. If the Linux configuration menu changes, execute the linux build target to recompile Linux.

```
$ make linux
```

If you change the configuration via *linux-menuconfig* and want to save it for the next compilation, you can use the subcommand linux-update-defconfig overrides the defconfig file specified by BR2_TARGET_KERNEL_CUSTOM_CONFIG_FILE. file specified by BR2_TARGET_KERNEL_CONFIG_FILE.

```
$ make linux-update-defconfig
```

Recompile Linux with the new configuration. Due to the large size of the Linux source code, it is not recommended to use linux-dirclean, just re-execute the reconfigure.

```
$ make linux-reconfigure
```

4.2 Linux kernel standalone compilation

The Linux kernel can be compiled independently of the SDK, and the development environment requires the mkimage command to be installed first. For Ubuntu users, the mkimage command can be obtained from installing the For Ubuntu users, you can get the mkimage command from installing the u-boot-tools package, or from the SDK.

```
$ sudo apt install u-boot-tools
or
$ cp [path/of/rts39xx_sdk_va.b.c/out/rts3918n_evb/host/usr/bin/mkimage] ~/
↪bin/
```

To compile Linux independently, first create a separate compilation workspace.

```
$ mkdir linux_work
$ cd linux_work
```

Copy the required source code and default configuration from the SDK.

```
$ cp -afr [path/of/rts39xx_sdk_va.b.c/platform/source/kernel/linux-6.6] .
$ cp -f [path/of/rts39xx_sdk_va.b.c/platform/board/rts3917/linux_6.6_
↪defconfig] linux_6.6/.config
```

Setting up the compilation environment

```
$ export PATH=$PATH:[path/of/rts39xx_sdk_va.b.c/toolchain/asdk-12.4.1-a7-EL-
↪6.6-u1.0-a32nh-linux-x86_64-241017/bin]
$ export ARCH=arm
$ export CROSS_COMPILE=arm-linux-
```

Execute make to generate uImage

```
$ make uImage -j4
```

When starting a new project, compiling linux independently, you usually need to customize your own device tree file, here rts3918n_cust.dts as an example. First create your own device tree file rts3918n_cust.dts using rts3918n_evb.dts as a template. add the device tree file to the device tree directory.

```
$ mv rts3918n_cust.dts arch/arm/boot/dts/realtek/
```

Select the device tree source file named rts3918n_cust.dts in the kernel option of menuconfig.

```
$ make menuconfig
Kernel  --->
    ...
    (realtek/rts3918n_cust) Device Tree Source file names
    ...
```

Once you have completed the above steps, you can compile with the new device tree.

5 Filesystem

5.1 Introduction to the Root Filesystem

The root directory of a Linux system is '/', and after the kernel boots the system mounts the device as the root directory, i.e. the root filesystem. This is the root filesystem. All system mount points, system configurations, etc. are located under the root filesystem. The root filesystem holds regular memory, flash or network filesystems, etc. under it. All applications, libraries, and other services need to be located under the root filesystem. The table *List of root filesystem directories* shows the directories under the root directory.

Table 5.1: List of root filesystem directories

directory	descriptions
sbin	An executable file that holds basic system commands
bin	An executable file that holds basic commands
dev	Device File Nodes
etc	System configuration files, including startup files
lib	Basic libraries such as C libraries, kernel modules
media	Temporary file system mount point (typically used to mount removable devices, such as SD cards)
mnt	Temporary file system mount points
proc	Virtual file system for exporting kernel and process information
sys	System equipment and files
tmp	Temporary document
usr	Practical Applications and Documentation
var	System logs and temporary files for services

5.2 OverlayFS

The OverlayFS file system is a widely used hierarchical file system with a simple implementation and good performance. With upper and lower merging, Same-name masking, copy-on-write, and other features. Currently, the SDK supports OverlayFS, which means viewing the entire file system from an application perspective. All are readable and writable.

5.2.1 Usage precautions for OverlayFS in RTS3917:

1. After configuring OverlayFS in the menuconfig, entering the root directory will reveal two additional directories: /rom and /overlay, where /rom contains Read-only partial files (i.e., ipcam_sdk/image/rootfs. The content in bin), while /overlay stores what the user has done. Modifications, here are a few examples:

   If the user creates a new file /bin/test, the bin/test file will be created in the /overlay directory.

   If the user deletes the file /bin/ls, a soft link to /bin/ls will be created in the /overlay directory (indicating that the file has been deleted).

   If the user modifies the file /etc/version, a version file will be created in the /overlay directory (storing the modified content).

2. All files in the file system directory are readable and writable, so the user's modifications include create, remove, and modify. Once almost everything changes, it won't disappear after a power outage, so it won't disappear after updating rootfs (since the modifications are stored in Flash). (In the middle, and separate from where rootfs is stored). If you do not need to save the previous modifications, please delete all content in the /overlay directory (which requires a restart to take effect). If you need to retain some of the modified content, such as /usr/conf, please keep the /overlay/usr/conf folder. Other files in the /overlay directory can be deleted.

5.3 SquashFS

The SquashFS file system is a read-only compressed file system designed for embedded systems and currently supports zlib, lzma, and lzo. Wait for the compression algorithm.

5.3.1 Create and Use the SquashFS File System

1. Configure the kernel and add SquashFS support

```
File systems --->
   [*] Miscellaneous filesystems --->
      <*> SquashFS 4.0 - Squashed file system support
      [*] Include support for XZ compressed file systems
```

2. Creating SquashFS Images

   Makefile will automatically call the mksquashfs tool to create SquashFS images. If you need to manually generate a SquashFS image file (using the lzma compression format), you can execute the following command:

```
$ $(HOST_DIR)/bin/mksquashfs $(TARGET_DIR) ./image/rootfs.bin -comp xz
```

   Where HOST_DIR can be . /out/rts3918n_base/host, and TARGET_DIR is the directory where rootfs is located.

5.4 JFFS2

JFFS2 is developed based on JFFS file system. As a practical file system for early embedded small devices, JFFS2 is equipped with read and write functions and has the following features:

- Store compressed files, the most important feature is to have read and write capabilities
- The disadvantage is that the mounting process scans the entire file system, so the mounting time is positively correlated with the partition size.

5.4.1 Creating And Using The JFFS2 Filesystem

1. Configure the kernel to add MTD_BLOCK support

```
Device Drivers --->
   <*> Memory Technology Device (MTD) support --->
      <*> Caching block device access to MTD devices
```

2. Configure the kernel to add JFFS2 support

```
File systems --->
   [*] Miscellaneous filesystems --->
      <*> Journalling Flash File System v2 (JFFS2) support
```

3. Creating a JFFS2 image

   SDK merges all partition images into a single file with the command *make pack* . You need to configure the paths of the mirrors before packing them. Specifically you need to edit the file **out/rts3918n_evb/merge.ini** . If you want to specify other mirrors, just modify the path directly.

   We assume that the default path is used and that the JFFS2 image is generated to the **image** directory. However, it is important to note that JFFS2 images are generated based on the **jffs2** directory. If this directory does not exist, the JFFS2 image will not be packed.

   To modify the JFFS2 image is also a simple matter of. Just modify the files in the **jffs2** directory.

4. Burn a merged image

   Please refer to *Firmware Burning* for details.

5. Using JFFS2 Partitions

   By default the JFFS2 partition will be mounted to the **/usr/conf** directory. If the files in this directory are modified, then the corresponding ones will be saved to the Flash Therefore the new data will not be lost after system reboot.

6. Configure mtd tools to add flash partition erase function (`flashcp` `, ``flash_erase`) support.

```
Target packages --->
   Filesystem and flash utilities --->
      [*] mtd, jffs2 and ubi/ubifs tools
      [*] flashcp
      [*] flash_erase
```

Note: When the customer gets the board, it may already have the corresponding system. When the newly updated version is partitioned differently from the old version, it may cause this error to be reported at startup **jffs2:: Magic bitmask 0x1985 not found at xxxxxxxx. jffs2_scan_eraseblock(): Magic bitmask 0x1985 not found at xxxxxxxx** . At this time, you need to erase and write the contents of the userdata

partition This is necessary to ensure that the system is running normally. First, use cat /proc/mtd to confirm the location of the MTD partition block device, and then use *flash_erase -j /dev/mtdX 0 0* to erase it. Where /dev/mtdX is the name of the userdata partition.

5.5 UBIFS

UBIFS is a new flash file system, considered to be the next generation of the JFFS2 file system, a file system used in high-capacity flash. UBIFS handles actions with the MTD device through the UBI subsystem. Like JFFS2, UBIFS is built on top of the MTD device. Thus it is not compatible with general block devices. Mainly used for Raw flash devices (typically Nand-flash), the UBIFS file system has the following characteristics:

- Predictability: UBIFS mount time, memory consumption, and I/O communication time are all independent of flash size, so UBIFS works better on large-capacity flash.
- Fast mounting: Unlike jffs2 which scans the storage media when mounting, UBIFS mounts in a few milliseconds regardless of the flash size.
- For UBIFS usage scenarios, there is usually only one module, Nand-Flash, and its interface to the controller is usually a specialized NAND FLASH interface.

5.5.1 Creating And Using The UBIFS Filesystem

Notes on using the UBIFS file system for the User data partition in NAND Flash are as follows:

1. Configure kernel to add NAND-Flash and UBIFS support

```
Device Drivers --->
   <*> Memory Technology Device (MTD) support --->
      <*> SPI NAND device support --->
         <*> Realtek Quad SPI Nand Flash Controller

Device Drivers --->
   <*> Memory Technology Device (MTD) support --->
      <*> Enable UBI - Unsorted block images --->

File systems --->
   <*> Miscellaneous filesystems --->
      <*> UBIFS file system support
```

2. Configure the partition file to add Nand-flash support, refer to the U-Boot chapter *Image file structure*

Note: If you want to use another capacity of NAND FLASH, or if you want to resize individual partitions. Please modify **kernel/linux-6.6/arch/arm/boot/dts/realtek/rts3918n_nand.dts** file.

3. Configure mtd tools

```
Target packages --->
   Filesystem and flash utilities --->
      [*] mtd, jffs2 and ubi/ubifs tools
      [*] ubiattach
      [*] ubiformat
      [*] ubimkvol
```

4. Burn merge image

   Please refer to *Firmware Burning* for details.

5. Using UBIFS Partitions

   The first boot requires the execution of mkubi.sh to execute **mkubi.sh /dev/mtdX** (the partition where userdata is located)

6. Reboot the RTS3917.

6 Image file structure

In U-Boot, partition images can be flashed individually, or they can be packed into a single image file (which we can call **linux.bin**) using *make pack* and flashed all at once.

This section mainly introduces the file structure of the linux.bin image file.

The structure of the linux.bin file can be referenced as follows: Fig. 6.1 .

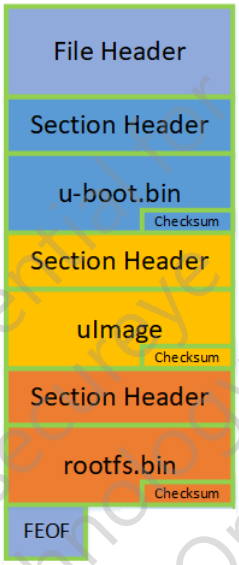

Fig. 6.1: linux.bin file structure

The header section of the file is fixed at 256 bytes, used to store basic information about the image. This part will not be flashed to the flash memory.

Note: The linux.bin generated by the default configuration in the SDK does not include uboot.bin. If you need linux.bin to include uboot.bin, you need to remove the comment regarding uboot.bin in the merge.ini file.

Each programming section includes: 4 bytes for the magic number, 8 bytes for the address, 8 bytes for the length, the data area, and 4 bytes for the checksum. (Refer to Section 2.4)

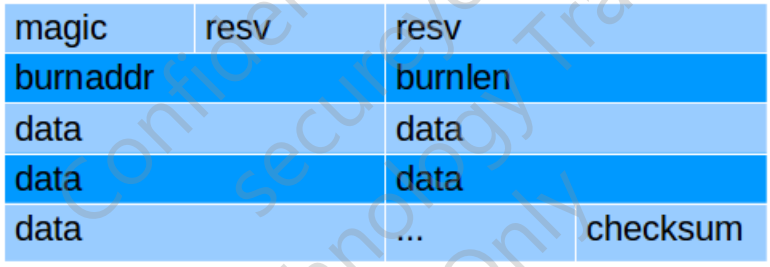

Fig. 6.2: Programming Partition Format

7 Developer's Guide

7.1 How to add compiled custom source packages

Adding a new source package to the SDK often requires only the following steps:

1. The source code for the package is placed in platform/source/<package_name>.
2. Add the Config.in and .mk menu files to the platform/package/<package_name> directory and write the

Config.in is used for menu configuration in the new menuconfig, with bool type as an example:

```
config MY_BOOLEAN
    bool "Enable Boolean Option"
    default y
    help
    This enables a boolean option for the feature.
```

The main purpose of .mk is to define the compilation rules, and the name of the mk file should be the same as the name of the package_name. make will automatically compile the source file and generate the final file according to the rules in the .mk file.

3. Include the Config.in of the <package_name> folder in the parent Config.in of <package_name>. That is, add source "$BR2_EXTERNAL_platform_PATH/package/<package_name>/Config.in" to the parent Config.in

4. If the package provides patches, the patch file can be placed in platform/patches/<package_name>.

The package menu file can be written with reference to other packages in the SDK such as kmod_rtl8188fu, whose buildroot related rules are defined with reference to

`Buildroot User's Manual Chapter 17 <https://buildroot.org/downloads/manual/manual.html#adding-packages>.`

The rules of buildroot are defined in the Buildroot User's Manual' chapter 17 <>_.

7.2 How to add a customized board profile

The SDK allows users to add customized boards using existing boards as templates, using the RTS3918N customized board as an example. RTS3918N EVB as a template. the RTS3918N EVB related files are in platform/board/rts3917. Copy it as a new directory, such as rts3918n_cust

```
$ cp -fr platform/board/rts3917 platform/board/rts3918n_cust
```

The files within rts3918n_cust need to be modified to fit the new board.

- busybox.config: Busybox defconfig file, no need to modify it manually, see the steps that follow

- uboot_2023.01_defconfig: U-Boot defconfig file, no need to modify it manually, see the following steps
- linux_6.6_defconfig: Linux kernel defconfig file, no need to modify it manually, see the following steps
- post_build.sh: Post-processing script specified by BR2_ROOTFS_POST_BUILD_SCRIPT
- post_image.sh: Post-processing script specified by BR2_ROOTFS_POST_IMAGE_SCRIPT
- rootfs_overlay: The rootfs overlay stores a few scattered files that are needed by rootfs but are not part of a package

Add a new Buildroot defconfig file to the platform/configs directory and modify its contents

```
$ cp -fr platform/configs/rts3918n_base_defconfig platform/configs/rts3918n_
↪cust_defconfig
$ sed 's%board/rts3917%board/rts3918n_cust%g' -i platform/configs/rts3918n_
↪cust_defconfig
```

Using the rts3918n_evb.dts device tree source file as a template, create the device tree file rts3918n_cust.dts for the new board. Add the created new device tree file to the device tree source file directory.

```
$ mv rts3918n_cust.dts platform/source/kernel/linux-6.6/arch/arm/boot/dts/
↪realtek/
```

Once the above changes are complete, you can run launch with the new buildroot defconfig

```
$ ./launch.sh rts3918n_cust_defconfig
$ cd out/rts3918n_cust
```

Further board adaptation is done using the following steps

Updating busybox.config

```
$ make busybox-menuconfig    #Make the necessary changes
$ make busybox-update-config
```

Update uboot_2023.01_defconfig

```
$ make uboot-menuconfig    #Make the necessary changes
$ make uboot-update-defconfig
```

Update linux_6.6_defconfig

```
$ make linux-menuconfig    #Make the necessary changes
$ make linux-update-defconfig
```

Update rts3918n_cust_defconfig

Change the device tree name BR2_LINUX_KERNEL_INTREE_DTS_NAME to realtek/rts3918n_cust in make menuconfig.

```
$ make menuconfig    #Make the necessary changes
$ make savedefconfig
```

All the adaptations have been completed and compilation is performed to generate a new image.

```
$ make
```